均值模糊

均值模糊
- 例程效果
- 原理讲解
  - 什么是图像均值模糊滤波
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置均值模糊参数
    - 图像处理与均值模糊滤波
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/23.rgb888_mean_blur.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像均值模糊滤波处理。



原理讲解

什么是图像均值模糊滤波

图像均值模糊滤波（Mean Blur）是一种简单的图像平滑技术，用于减少图像中的噪声并模糊细节。它通过对图像中的每个像素及其周围像素进行平均计算来实现平滑效果。均值模糊是图像处理中最基础的滤波方法之一，适用于快速去噪。

- **操作原理**：均值模糊使用一个卷积核（通常是矩形或正方形），对图像上的每个像素及其邻域内的像素值进行平均计算。核内的所有位置具有相同的权重（即均匀分布），与高斯滤波不同，均值模糊不考虑距离中心的权重变化。结果是每个像素被替换为其邻域内的平均值，从而平滑图像。
- **效果**：均值模糊能够有效去除图像中的高频噪声（如随机噪点），但可能会导致图像细节（如边缘）丢失，产生较为明显的模糊效果。相比高斯滤波，均值模糊更容易引入方块状伪影，因为它不考虑像素间的距离权重差异。
- **应用场景**：均值模糊常用于图像去噪、预处理阶段的平滑处理（如在边缘检测前减少噪声干扰）以及需要快速模糊效果的场景。由于其计算简单，适合在计算资源有限的嵌入式设备中使用。

在均值模糊中，卷积核的大小（必须为奇数）会显著影响结果：较大的核会导致更强的模糊效果，但计算量也会增加，同时可能丢失更多细节。

代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import _thread
import cv_lite                # cv_lite扩展模块 / cv_lite extension (includes
mean_blur API)
import ulab.numpy as np       # ulab 数组模块 / NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(width=1280, height=960)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to
RGB888
```

- 初始化摄像头，设置分辨率为 1280x960（帧率未指定，使用默认值）。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的均值模糊滤波）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=100)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=100` 设置图像传输质量为最高。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置均值模糊参数

```
kernel_size = 3
clock = time.clock()    # 启动帧率计时器 / Start FPS timer
```

- `kernel_size`：均值模糊的卷积核尺寸，必须为奇数（如 3、5、7等），值越大模糊效果越强，但计算量增加。本代码中设置为 3，属于较小的核，适合轻度平滑和去噪。
- `clock`：用于计算帧率。

图像处理与均值模糊滤波

```
while True:
    clock.tick()  # 开始计时 / Start timing

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray (HWC)

    # 应用均值模糊滤波 / Apply mean blur using cv_lite
    denoised_np = cv_lite.rgb888_mean_blur_fast(
        image_shape,
        img_np,
        kernel_size
    )

    # 构造图像用于显示 / Wrap processed image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                          alloc=image.ALLOC_REF, data=denoised_np)

    # 显示图像 / Display the blurred image
    Display.show_image(img_out)

    # 清理内存并打印帧率 / Collect garbage and show FPS
    gc.collect()
    print("mean blur fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **均值模糊处理**：调用 `cv_lite.rgb888_mean_blur_fast()` 函数执行均值模糊滤波操作，返回处理后的图像 NumPy 数组。
- **图像包装和显示**：将处理结果包装为 RGB888 格式图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `kernel_size`：均值模糊的卷积核尺寸，值越大模糊效果越强，适合处理更高水平的噪声或需要更平滑的图像；必须使用奇数（如 3、5、7等），建议从 3 开始测试，逐步增加到 7 或 9 这样的较大值以观察效果和性能影响。较大的核会显著增加计算量，可能降低帧率。